

Event Rakshak

Event Insurance Management Platform

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Full System Documentation

Document Attribute	Details
Document Title	Event Rakshak SRS — Full-Stack Event Insurance Platform
Status	Final — For Review
Technology Stack	Angular 21 Spring Boot 3.4 H2 Database
Architecture Pattern	RESTful API with JWT Stateless Authentication
Classification	Internal Technical Documentation

Table of Contents

Table of Contents..... 2

Section 1: Introduction 5

 1.1 Purpose of This Document..... 5

 1.2 Project Scope 5

 1.3 Definitions, Acronyms, and Abbreviations..... 6

 1.4 Document Conventions 6

 1.5 Intended Audience 7

 1.6 Comprehensive Product Overview..... 7

Section 2: System Architecture..... 8

 2.1 Architectural Overview 8

 2.2 Frontend Architecture (Angular 21)..... 8

 2.2.1 Core Module 8

 2.2.2 Features Module..... 8

 2.2.3 Routing and Role Guards 8

 2.3 Backend Architecture (Spring Boot 3.4)..... 8

 2.3.1 Controller Layer 9

 2.3.2 Service Layer..... 9

 2.3.3 Repository Layer..... 9

 2.3.4 Security Configuration 9

 2.4 Database Architecture (H2 File-Based)..... 9

 2.5 RESTful API Contract..... 10

 2.6 JWT Authentication Flow..... 10

Section 3: Feature Specifications by User Role..... 12

 3.1 Customer Persona — Feature Specifications 12

 3.1.1 Authentication — Registration and Login 12

 3.1.2 Home and Landing Page 12

 3.1.3 Event Management..... 12

 3.1.4 Policy Marketplace..... 14

 3.1.5 Subscription Workflow 14

 3.1.6 Payment Gateway (Simulated) 14

 3.1.7 Claims Center 14

 3.2 Underwriter Persona — Feature Specifications 15

3.2.1 Underwriter Dashboard.....	15
3.2.2 Subscription Review Interface	15
3.3 Claims Officer Persona — Feature Specifications	15
3.3.1 Claims Dashboard	15
3.3.2 Claim Processing Interface.....	15
3.4 Administrator Persona — Feature Specifications.....	15
3.4.1 Admin Dashboard	16
3.4.2 User Management	16
3.4.3 Policy Management	16
Section 4: The Risk Assessment Engine — Deep Dive.....	17
4.1 Architectural Overview of the Risk Engine	17
4.2 The Strategy Pattern Implementation.....	17
4.3 Music Concert Risk Strategy — Mathematical Model	17
4.4 Corporate Conference Risk Strategy — Mathematical Model.....	18
4.5 Premium Calculation Formula	19
4.6 WeatherRiskService — Live Meteorological Integration	19
4.7 Risk Engine Input/Processing/Output Specification	19
4.7.1 Input Specification	19
4.7.2 Processing Steps.....	20
4.7.3 Output Specification.....	20
Section 5: Data Design and Persistence Strategy	21
5.1 Entity Relationship Overview.....	21
5.2 Entity Specifications	21
5.2.1 User Entity (users table)	21
5.2.2 Role Entity (roles table)	22
5.2.3 Event Entity (events table).....	22
5.2.4 Policy Entity (policies table).....	22
5.2.5 PolicySubscription Entity (policy_subscriptions table).....	23
5.2.6 Claim Entity (claims table)	24
5.2.7 Notification Entity (notifications table).....	25
5.3 Database Normalization Strategy.....	25
5.4 Initial Seed Data Catalog.....	25
Section 6: Non-Functional Requirements	27
6.1 Security Requirements	27
6.1.1 Authentication Security	27
6.1.2 Role-Based Access Control (RBAC)	27

6.1.3 Input Validation	27
6.2 Performance Requirements	27
6.3 Availability and Reliability	28
6.4 H2 Database — Disaster Recovery Considerations	28
6.5 Scalability and Maintainability	28
6.6 Usability Requirements	28
Appendix A: Technology Stack Summary.....	29
Appendix B: UML Diagram Placeholders Summary	Error! Bookmark not defined.
Appendix C: Complete API Endpoint Reference	29
Appendix D: Database DDL Export Script (H2)	Error! Bookmark not defined.
D.1 Database Administrator Account	Error! Bookmark not defined.
D.2 Table Definitions	Error! Bookmark not defined.
D.3 Primary Key Constraints	Error! Bookmark not defined.
D.4 Unique Constraints	Error! Bookmark not defined.
D.5 Foreign Key Constraints	Error! Bookmark not defined.
Section 7: Testing Specification.....	32
7.4 Unit Testing Specification.....	32
7.4.1 Backend Unit Testing (Java)	32
A. Testing Philosophy — Mocking Strategy.....	32
B. Core Service Layer Testing.....	33
C. Entity and DTO Validation Testing	34
7.4.2 Frontend Unit Testing (Angular 21)	35
A. Component Testing	35
B. Reactive Form Testing	35
C. Service and Guard Testing	35
7.4.3 Sample Unit Test Registry.....	37
7.4.4 Test Code Snippets.....	39
Snippet 1 — JUnit 5: RiskCalculationServiceTest.....	39
Snippet 2 — Mockito: ClaimServiceTest	Error! Bookmark not defined.
Snippet 3 — Jasmine: LoginComponent spec (Angular 21) ..	Error! Bookmark not defined.

Section 1: Introduction

1.1 Purpose of This Document

This Software Requirements Specification (SRS) constitutes the authoritative, comprehensive technical reference document for the Event Rakshak platform — a purpose-built, full-stack event insurance management system. This document has been prepared to serve as both a functional blueprint and a technical specification, bridging communication between business stakeholders, software architects, development teams, quality assurance engineers, and operational personnel.

The primary objective of this SRS is to define, with precision and completeness, the behavioral and non-behavioral requirements of the Event Rakshak system. It captures the expected behavior of each system component, the data models that persist business state, the security framework governing access, the integration points with third-party services, and the non-functional constraints that govern performance, reliability, and maintainability.

This document is intended to serve as a living reference throughout the software development lifecycle (SDLC), from initial design through implementation, testing, deployment, and ongoing maintenance. Any implementation decisions that deviate from specifications described herein must be formally reviewed and documented as change requests.

1.2 Project Scope

Event Rakshak is a comprehensive digital platform that serves the unique insurance needs of the events industry. Traditional insurance management systems are often generic and require significant customization for event-specific use cases. Event Rakshak addresses this gap by providing a domain-specific platform that natively understands event types, risk factors, and the operational workflow from event creation through insurance policy activation and eventual claim resolution.

The platform supports four distinct user personas — Customers, Underwriters, Claims Officers, and Administrators — each with a tailored interface and a precisely scoped set of permissions. This separation of concerns is enforced both at the UI routing level and the backend API security layer, ensuring that no user can access data or execute operations outside their designated role.

The system scope encompasses the following high-level functional domains:

- Identity and Access Management: Multi-role user registration, login, and JWT-based session management.
- Event Portfolio Management: Full CRUD lifecycle for event records, including rich metadata capture for risk profiling.
- Insurance Policy Catalogue: Administrative management of policy templates categorized by event domain.
- Risk Assessment Engine: Automated, multi-factor risk calculation using domain strategies and live meteorological data.
- Subscription and Underwriting Workflow: End-to-end policy application lifecycle from customer request to underwriter approval and payment.
- Claims Processing Workflow: Structured claim filing, evidence submission, review, and adjudication.
- Notification System: Real-time event-driven alerts for workflow state transitions.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification — this document
JWT	JSON Web Token — a compact, URL-safe token standard for representing identity claims
RBAC	Role-Based Access Control — access management model based on user roles
API	Application Programming Interface — the contract between frontend and backend systems
ORM	Object-Relational Mapping — technique to map Java entity objects to database tables
JPA	Jakarta Persistence API — Java standard for ORM with Hibernate as the provider
CRUD	Create, Read, Update, Delete — the four basic database operations
SPA	Single-Page Application — a web app that loads once and dynamically updates
DI	Dependency Injection — Spring's mechanism for providing component dependencies
DTO	Data Transfer Object — a plain object used to carry data between process layers
NOC	No Objection Certificate — a safety compliance document required for certain event types
ERD	Entity Relationship Diagram — a visual representation of the database schema
WeatherAPI	Third-party REST API at weatherapi.com providing real-time meteorological data
H2	An open-source, Java-based embedded/server-mode relational database engine
DDL	Data Definition Language — SQL commands for defining database schema
SDLC	Software Development Lifecycle — the structured process for software creation and maintenance

1.4 Document Conventions

This document adheres to the following conventions to ensure consistency and readability across all sections:

- Monospace or bold formatting is used for code references, class names, service identifiers, and API endpoint paths.

- Placeholder markers such as [INSERT FIGURE X: ...] denote locations where diagrams or visual artifacts are to be embedded by the documentation team.
- Risk factor values expressed as percentages (e.g., +2.0%) refer to additive adjustments to the base risk percentage score computed by the Risk Assessment Engine.
- All API paths are expressed relative to the base URL of the backend server. For local development, the default base is `http://localhost:8080`.
- Database field types are described using Hibernate/JPA annotations where relevant.
- User roles are expressed in their canonical Spring Security format: `ROLE_ADMIN`, `ROLE_CUSTOMER`, `ROLE_UNDERWRITER`, `ROLE_CLAIMS_OFFICER`.

1.5 Intended Audience

This document is intended for the following audiences:

- Software Engineers and Architects: For implementation guidance across frontend, backend, and database layers.
- Quality Assurance Engineers: As the basis for test case and test plan development.
- Project Managers: For scope verification and milestone tracking.
- Business Analysts: For validating that functional requirements reflect business needs.
- Operations and DevOps Teams: For deployment, monitoring, and database management procedures.
- Security Auditors: For RBAC verification, token lifecycle assessment, and penetration testing scoping.

1.6 Comprehensive Product Overview

Event Rakshak was conceived to address a well-documented challenge in the events industry: the prohibitively complex, paper-intensive, and opaque process of obtaining insurance coverage for events. Traditional insurance procurement involves lengthy underwriting assessments, manual data collection, and non-standardized claim processes. Event Rakshak digitizes and streamlines this entire lifecycle on a single, unified platform.

The platform models the real-world insurance process faithfully. A Customer registers, creates their event with full operational details, and browses available policy templates tailored to their event domain (e.g., Music Concert, Corporate Conference). They apply for a subscription, triggering an automated risk assessment. An Underwriter then reviews this risk profile, has the option to override the system-calculated premium, and approves or rejects the application. Upon approval and payment, the policy becomes active. Should an insured incident occur, the customer can file a claim with documentary evidence. A Claims Officer then reviews the claim and adjudicates the payout.

This end-to-end workflow is supported by a modern technology stack. The Angular 21 frontend provides a responsive, role-aware single-page application. The Spring Boot 3.4 backend exposes a secure, RESTful API layer with fine-grained access control. The H2 file-based database persists all domain data with full relational integrity enforced at the ORM level. WeatherAPI.com integration enriches risk assessments for outdoor events with live meteorological data.

Section 2: System Architecture

2.1 Architectural Overview

Event Rakshak implements a three-tier client-server architecture: a presentation tier (Angular SPA), a business logic tier (Spring Boot REST API), and a data persistence tier (H2 relational database). This layered architecture ensures that each tier has clear responsibilities and can be evolved independently, promoting maintainability and testability.

The frontend communicates exclusively with the backend via HTTP/HTTPS RESTful calls. There is no direct connection between the Angular application and the database. All data access is mediated through the API layer, which enforces authentication, authorization, business logic, and data validation before any persistence operation occurs.

2.2 Frontend Architecture (Angular 21)

The frontend is implemented as an Angular 21 Single-Page Application. Angular's component-based architecture provides a natural mapping between UI concerns and distinct functional areas of the application. The application follows the Features-Core architectural pattern, which organizes code into shared infrastructure (core/) and domain-specific features (features/).

2.2.1 Core Module

The core/ module houses infrastructure-level artifacts that are consumed globally across the application. This includes authentication services responsible for token storage and validation, HTTP interceptors that automatically inject Bearer tokens into outgoing requests, route guards that redirect unauthenticated or unauthorized users, and the notification service that polls for backend alerts. Services declared in the core module are provided at the root injector level (providedIn: 'root') to ensure singleton instances across the entire application, preventing state divergence that could arise from multiple service instantiations.

2.2.2 Features Module

The features/ module organizes the domain-specific components into logical groupings. Each feature represents a bounded context of the application — for example, the events feature encapsulates event listing, event creation, and event detail views. The policy feature handles the marketplace display and detail views. The subscriptions feature manages the application workflow and payment gateway integration. The claims feature covers claim filing and status tracking. Each feature module declares its own routes, which are lazily loaded by the Angular Router to reduce the initial application bundle size and improve first-load performance.

2.2.3 Routing and Role Guards

Angular RouterModule is configured with lazy-loaded feature modules to optimize initial load performance. Each protected route is guarded by the RoleGuard, which reads the decoded JWT payload from local storage and evaluates whether the authenticated user's role satisfies the route's required roles. Unauthorized navigation attempts are redirected to a designated 403 error component or to the login page if no token is present.

2.3 Backend Architecture (Spring Boot 3.4)

The backend is a Spring Boot 3.4 application compiled with Java and built using Apache Maven. The application follows a layered architecture within the Spring ecosystem: Controller Layer to Service Layer to Repository Layer to Entity/Domain Layer. This strict directionality of dependency (each layer depends only on the layer below it) enforces a clean separation of concerns and enables independent unit testing of each layer.

2.3.1 Controller Layer

The controller layer is responsible for request routing, HTTP verb mapping, request validation, and response serialization. Each controller is a Spring `@RestController` annotated class that maps incoming HTTP requests to service method invocations. Controllers do not contain business logic; they delegate entirely to the service layer. Response payloads are serialized to JSON by the Jackson library, which is auto-configured by Spring Boot. `@RequestBody` annotations trigger automatic deserialization of JSON request bodies into DTO objects, while `@PathVariable` and `@RequestParam` extract URL and query parameters respectively.

2.3.2 Service Layer

The service layer contains all business logic. This includes orchestrating multi-step workflows (such as the subscription lifecycle), calling repository methods, applying business rule validations, and interfacing with external services (such as WeatherAPI). Services are annotated with `@Service` and are injected into controllers via constructor-based dependency injection. `@Transactional` annotations ensure that multi-step database operations either complete atomically or roll back completely upon failure, maintaining data integrity across complex workflow transitions.

2.3.3 Repository Layer

The repository layer provides an abstraction over database operations using Spring Data JPA. Each entity has a corresponding repository interface that extends `JpaRepository` or `CrudRepository`, providing standard CRUD operations out of the box, as well as support for custom JPQL queries declared as interface method signatures using the `@Query` annotation. Spring Data JPA automatically generates the implementation at runtime via dynamic proxy generation, eliminating the need for boilerplate DAO implementation code.

2.3.4 Security Configuration

Spring Security is configured to use stateless JWT authentication. A `JwtAuthenticationFilter` intercepts every incoming request, extracts the Bearer token from the Authorization header, validates the token's signature and expiry, and populates the Spring `SecurityContext` with the authenticated user's principal and granted authorities. The `SecurityConfig` class defines which endpoints are publicly accessible (e.g., `/api/auth/**`) and which require authentication, using the `HttpSecurity` fluent API to configure endpoint-level access rules.

2.4 Database Architecture (H2 File-Based)

The data persistence tier uses the H2 relational database engine in file-based persistent mode. The database is named `insurance_db` and is stored on the filesystem adjacent to the application binary. H2 was selected for its zero-configuration deployment model, full SQL standard compliance, and suitability for development and demonstration environments.

Hibernate ORM (as the JPA provider) manages all DDL operations. With `spring.jpa.hibernate.ddl-auto=update`, Hibernate automatically creates tables for new entities and adds new columns to

existing tables on application startup, while preserving existing data. This provides a seamless schema evolution experience during development without requiring manual migration scripts. The H2 web console is enabled for development environments, accessible at /h2-console, enabling direct database inspection and ad-hoc SQL query execution during development and debugging.

The data.sql seed script executes after Hibernate schema creation (deferred datasource initialization is enabled) and populates the database with standard roles, a default administrator account, and the initial insurance policy catalogue. This ensures a consistent, functional baseline database state upon first deployment.

2.5 RESTful API Contract

The frontend and backend communicate via a well-defined RESTful API contract. All requests and responses use JSON as the data interchange format. HTTP status codes are used semantically: 200/201 for successful operations, 400 for validation failures, 401 for authentication failures, 403 for authorization failures, 404 for missing resources, and 500 for unexpected server errors. The following table summarizes the primary API endpoint namespaces:

Namespace	HTTP Methods	Primary Consumer	Description
/api/auth/**	POST	All Users (Public)	Login and registration endpoints — no authentication required
/api/events/**	GET, POST, PUT, DELETE	Customer	Full CRUD operations on a customer's event portfolio
/api/policies/**	GET	Customer	Read-only policy catalogue browsing
/api/subscriptions/**	GET, POST	Customer	Policy subscription applications and status tracking
/api/payments/**	POST	Customer	Simulated premium payment processing
/api/claims/**	GET, POST	Customer	Claim filing and status retrieval
/api/notifications/**	GET, PUT	All Authenticated	Notification history and read-status management
/api/underwriter/**	GET, PUT	Underwriter	Subscription review, premium override, approval/rejection
/api/claims-officer/**	GET, PUT	Claims Officer	Claim queue management and adjudication
/api/admin/users/**	GET, POST, PUT	Admin	User account management and role assignment
/api/admin/policies/**	GET, POST, PUT, DELETE	Admin	Policy template CRUD operations

2.6 JWT Authentication Flow

The authentication mechanism is stateless, token-based, and implemented using the JSON Web Token (JWT) standard. The complete authentication lifecycle is described as follows:

1. The user submits credentials (email + password) to the POST /api/auth/login endpoint.

2. The AuthController delegates to the AuthService, which calls Spring Security's AuthenticationManager to validate credentials against the stored BCrypt-hashed password.
3. Upon successful authentication, the JwtService generates a signed JWT containing the user's email (subject), roles, and an expiration timestamp.
4. The token is returned to the client in the login response payload.
5. The Angular AuthInterceptor stores the token in browser local storage and automatically injects it as a Bearer token in the Authorization header of all subsequent outgoing HTTP requests.
6. For each protected API call, the JwtAuthenticationFilter on the backend validates the token signature, checks expiry, extracts the user identity, and populates the SecurityContext before the request reaches the controller.
7. If the token is expired, malformed, or absent, the security filter returns a 401 Unauthorized response and the Angular RoleGuard redirects the user to the login page.

Section 3: Feature Specifications by User Role

3.1 Customer Persona — Feature Specifications

The Customer is the primary end-user of the Event Rakshak platform. A Customer represents an event organizer or promoter who seeks insurance coverage for their events. The Customer persona has access to a rich set of features spanning event management, policy discovery, insurance application, payment processing, and claims management.

3.1.1 Authentication — Registration and Login

A prospective Customer accesses the publicly available registration form at the `/register` route. The registration form collects: Full Name, Email Address, Password (with confirmation), Company/Organization Name, and Phone Number. Angular Reactive Forms enforce field-level validation including email format verification, password strength requirements (minimum 8 characters), and required field constraints. Upon successful form submission, the frontend calls `POST /api/auth/register` with the collected data. The backend creates a new User entity, hashes the password using BCrypt, assigns the `ROLE_CUSTOMER` authority, and persists the record. Upon subsequent visits, the Customer uses the login form at `/login`. A successful login response delivers a JWT that the Angular AuthInterceptor stores and automatically injects into all subsequent API calls. Failed login attempts are surfaced as inline form error messages without revealing whether the email or password was the incorrect credential.

3.1.2 Home and Landing Page

The Customer's home page, accessible at `/home` after authentication, provides a contextual dashboard summarizing the customer's engagement with the platform. It prominently displays quick-action tiles for creating a new event, browsing policies, and reviewing active subscriptions. A notification badge in the header indicates the count of unread alerts, drawing attention to pending workflow transitions. The landing page also serves marketing content describing Event Rakshak's value proposition for visitors who are not yet authenticated.

3.1.3 Event Management

The event management feature is the foundational component of the Customer's workflow, as events are the primary unit of coverage for all insurance operations. The event listing view at `/events` displays all events owned by the authenticated customer in a tabular format with pagination support. The create-event form at `/events/create` captures comprehensive event metadata required for risk profiling:

Field Name	Data Type	Validation Rules	Risk Relevance
Event Name	String	Required, 3-100 characters	Administrative identifier — no direct risk impact
Event Type	Enum	Required; MUSIC_CONCERT or CORPORATE_CONFERENCE	Determines which risk strategy class is invoked
Event Date	Date	Required; must be a future date	Used for weather API forecast lookup

Field Name	Data Type	Validation Rules	Risk Relevance
Location / Venue	String	Required; city-level granularity	Used for WeatherAPI geographic query parameter
Budget (INR)	Decimal	Required; positive value	Base figure for premium calculation formula
Expected Attendees	Integer	Required; positive integer	Crowd threshold factor: >1000 (Music) / >500 (Corporate)
Is Outdoor Event	Boolean	Required checkbox	Triggers WeatherRiskService integration if true
Venue Type	String	Optional descriptive field	Context for underwriter review — no direct risk formula impact
Alcohol Served	Boolean	Optional; Music concerts only	Direct +1.0% risk factor for Music Concert strategy
Temporary Stage	Boolean	Optional; Music concerts only	Direct +0.5% risk factor for Music Concert strategy
Fireworks / Pyrotechnics	Boolean	Optional; Music concerts only	Direct +1.5% risk factor for Music Concert strategy
Celebrity Involvement	Boolean	Optional; Music concerts only	Direct +2.0% risk factor for Music Concert strategy
High-Risk Geographic Location	Boolean	Optional; both domains	Direct +1.5% risk factor applied by relevant strategy
Security Level	Enum	Optional; LOW, MEDIUM, HIGH	LOW level adds +1.0% in Music Concert strategy
Fire NOC Obtained	Boolean	Critical compliance field	Missing NOC: +3.0% Music Concert / +2.5% Corporate
Metal Detectors Present	Boolean	Optional; Music concerts only	Risk mitigation: -0.5% in Music Concert strategy
Temporary Booths / Structures	Boolean	Optional; Corporate only	Direct +1.0% risk factor for Corporate Conference strategy
High-Value Equipment Assets	Boolean	Optional; Corporate only	Direct +2.0% risk factor for Corporate Conference strategy
Emergency Preparedness Level	Enum	Optional; LOW, MEDIUM, HIGH	LOW level adds +2.0% in Corporate Conference strategy
CCTV Surveillance Present	Boolean	Optional; Corporate only	Risk mitigation: -0.5% in Corporate Conference strategy

3.1.4 Policy Marketplace

The policy marketplace, accessible at /policies, presents all active insurance policy templates to the Customer. The marketplace implementation fetches data from GET /api/policies and renders policy cards organized by domain. Each policy card displays the Policy Name, Description, Base Rate expressed as a percentage of coverage, Maximum Coverage Amount, and a visual breakdown of covered risk types (weather, theft, fire, cancellation) rendered as badge-style indicators. Customers can filter the marketplace by event domain (Music / Corporate) and by coverage type. Selecting a policy reveals a detail view with the full specification and an Apply Now call-to-action initiating the subscription workflow.

3.1.5 Subscription Workflow

The subscription workflow is the most complex multi-step process in the Customer persona. It begins when the customer selects a policy and an associated event and submits an insurance application via POST /api/subscriptions. The backend's PolicySubscriptionService orchestrates the following automated sequence:

8. Retrieve the event record and associated policy template from the database.
9. Invoke the RiskCalculationService to compute the domain-specific risk percentage based on event metadata.
10. If the event is marked as outdoor, invoke the WeatherRiskService to fetch live meteorological data and compute an additional weather risk component.
11. Combine domain risk and weather risk into a total risk percentage.
12. Calculate the premium amount by applying the total risk percentage to the policy's base rate and the event budget.
13. Persist the PolicySubscription entity with status PENDING_REVIEW and assign to the underwriter queue.
14. Generate a Notification for the customer confirming receipt of the application.

3.1.6 Payment Gateway (Simulated)

Upon underwriter approval, the subscription status transitions to PAYMENT_PENDING and the customer is prompted to complete payment through the simulated payment gateway at /payment/:subscriptionId. The gateway interface presents the premium amount, policy name, and coverage details. The customer submits a dummy transaction, triggering POST /api/payments/:subscriptionId. The backend validates that the subscription is in PAYMENT_PENDING state, marks is_paid as true, and transitions status to ACTIVE. A confirmation notification is generated.

3.1.7 Claims Center

The Claims Center at /claims lists all claims filed by the authenticated customer. For each active (status=ACTIVE) subscription, the customer can file a new claim via the form at /claims/file/:subscriptionId. The claim form captures: Incident Description, Claim Amount Requested, Date of Incident, and an evidence document upload. The backend's ClaimService validates the claim against subscription coverage terms (checking covered risk types and maximum coverage limits) before persisting a new Claim entity with status PENDING. Filed claims can be tracked in the claims list view where status updates from Claims Officers appear in real time via the notification system.

3.2 Underwriter Persona — Feature Specifications

The Underwriter is a financial professional responsible for assessing insurance risk and making coverage decisions. Within Event Rakshak, the Underwriter's role is scoped to the subscription review and approval workflow. Underwriters cannot access claim management functions, and they cannot view or modify other users' data outside of assigned subscriptions.

3.2.1 Underwriter Dashboard

The Underwriter dashboard, accessible at `/underwriter/dashboard`, presents a prioritized queue of `PolicySubscription` records in `PENDING_REVIEW` or `UNDER_REVIEW` status. The dashboard renders a summary table with columns: Subscription ID, Customer Name, Event Name, Event Date, Event Type, Calculated Risk Percentage, System-Calculated Premium, and Submission Date. The data is retrieved from `GET /api/underwriter/subscriptions`. The dashboard auto-refreshes based on notification polling to surface newly assigned subscriptions in real time.

3.2.2 Subscription Review Interface

Selecting a subscription navigates to the detailed review interface at `/underwriter/review/:subscriptionId`. This view provides the underwriter with: full event metadata, the computed domain risk breakdown (factor-by-factor), the weather risk component with raw API values, the system-calculated premium amount, a premium override input field, and Approve/Reject action buttons each requiring a mandatory reason note.

The Approve action calls `PUT /api/underwriter/subscriptions/:id/approve` with the optionally overridden premium. The Reject action calls `PUT /api/underwriter/subscriptions/:id/reject` with the rejection reason. Both actions trigger notifications to the relevant customer.

3.3 Claims Officer Persona — Feature Specifications

The Claims Officer is responsible for adjudicating insurance claims filed by customers. The Claims Officer has view-only access to claim details and the ability to approve or reject claims. They cannot modify policies, subscriptions, or user accounts.

3.3.1 Claims Dashboard

The Claims dashboard at `/claims-officer/dashboard` displays all claims in `PENDING` status across all customers. The queue table presents: Claim ID, Customer Name, Policy Name, Incident Date, Claimed Amount, and Submission Date. The data is fetched from `GET /api/claims-officer/claims`. Dashboard rows are color-coded by claimed amount to visually prioritize high-value claims requiring escalated review attention.

3.3.2 Claim Processing Interface

The claim processing interface at `/claims-officer/review/:claimId` provides the full claim context including: the parent subscription details, the associated event record, the incident description, the evidence document download link, and the requested claim amount. The Claims Officer may approve the claim (recording an approved payout amount via `PUT /api/claims-officer/claims/:id/approve`) or reject it (recording a mandatory rejection reason via `PUT /api/claims-officer/claims/:id/reject`). Both actions trigger customer notifications.

3.4 Administrator Persona — Feature Specifications

The Administrator persona has the broadest access scope in the system. Administrators are responsible for platform configuration, user governance, and policy catalogue management. Administrative functions are restricted to users bearing the `ROLE_ADMIN` authority.

3.4.1 Admin Dashboard

The Admin dashboard at `/admin/dashboard` provides a system-wide metrics overview including: Total Registered Users by role, Total Events Created, Total Policy Subscriptions by status, Total Claims Filed by status, and Total Premium Revenue (sum of premiums for `ACTIVE` subscriptions). These metrics are computed server-side and returned by `GET /api/admin/metrics`.

3.4.2 User Management

The user management interface at `/admin/users` displays a searchable, paginated directory of all registered users. For each user, the administrator can: activate or deactivate the account (toggling the `enabled` flag), assign or remove roles, and view the user's subscription and claims history. These operations map to `PUT /api/admin/users/:id/status` and `PUT /api/admin/users/:id/roles` respectively.

3.4.3 Policy Management

The policy management interface at `/admin/policies` enables full CRUD operations on insurance policy templates. Creating a new policy requires: Policy Name (unique), Domain (`MUSIC` or `CORPORATE`), Description, Base Rate (percentage decimal), Maximum Coverage Amount, and boolean coverage flags for theft, weather, fire, and cancellation. These templates form the catalogue displayed in the Customer Policy Marketplace.

Section 4: The Risk Assessment Engine — Deep Dive

4.1 Architectural Overview of the Risk Engine

The Risk Assessment Engine is the technical core of Event Rakshak's value proposition. It is responsible for transforming raw event metadata into a quantified risk percentage that drives the insurance premium calculation. The engine is implemented across two primary service classes: RiskCalculationService and WeatherRiskService. Together, these services implement a Strategy design pattern to achieve domain-specific risk computation, augmented by real-time meteorological intelligence.

The risk engine produces a composite risk score composed of two independent components: a domain risk percentage (computed from event characteristics) and a weather risk percentage (computed from live weather data for outdoor events). These components are summed to produce the total risk percentage, which is then used by the PolicySubscriptionService to calculate the final premium amount billed to the customer.

4.2 The Strategy Pattern Implementation

The Strategy pattern is employed to enable domain-specific risk calculation without polluting the RiskCalculationService with conditional branching logic. The pattern defines a common IRiskStrategy interface with a single calculateRisk(Event event) method returning a double (the domain risk percentage). Concrete strategy implementations — MusicConcertRiskStrategy and ConferenceRiskStrategy — provide domain-specific logic.

The RiskCalculationService acts as the strategy context. Upon receiving a risk calculation request, it reads the event_type field of the Event entity and selects the appropriate strategy implementation via a factory lookup map. The selected strategy is then invoked with the full Event object, which carries all risk-relevant boolean flags and metadata fields collected during event creation.

This architecture provides several engineering advantages: new event domains can be supported by implementing a new strategy class without modifying existing service logic; unit tests can be written for each strategy in isolation; and the strategy selection logic is centralized and auditable.

4.3 Music Concert Risk Strategy — Mathematical Model

The MusicConcertRiskStrategy computes a domain risk percentage by evaluating each of the following risk factors against the event metadata. All factors are additive adjustments to a base risk of 0.0%. The strategy iterates through each factor, checks the corresponding boolean field or threshold condition on the Event entity, and accumulates the risk delta:

Risk Factor	Risk Delta	Type
Large Crowd (Attendees > 1,000)	+2.0%	Risk Increase
Outdoor Venue (isOutdoor = true)	+1.0%	Risk Increase
Alcohol Served at Event	+1.0%	Risk Increase

Risk Factor	Risk Delta	Type
Temporary Stage Structure	+0.5%	<i>Risk Increase</i>
Fireworks or Pyrotechnics Present	+1.5%	<i>Risk Increase</i>
Celebrity Involvement	+2.0%	<i>Risk Increase</i>
High-Risk Geographic Location	+1.5%	<i>Risk Increase</i>
Low Security Level	+1.0%	<i>Risk Increase</i>
No Fire NOC Obtained (Critical Compliance Failure)	+3.0%	<i>Risk Increase</i>
Metal Detectors Deployed (Risk Mitigation)	-0.5%	<i>Mitigation</i>

Maximum possible Music Concert domain risk (all positive factors, no mitigation): +13.5%. Minimum with mitigation: 0.0% (event with no risk factors). The computed domain risk percentage is a floating-point value rounded to two decimal places before storage on the PolicySubscription entity.

4.4 Corporate Conference Risk Strategy — Mathematical Model

The ConferenceRiskStrategy applies to events of type CORPORATE_CONFERENCE. The risk factor model reflects the distinct risk profile of corporate gatherings, which prioritize asset protection and liability management over crowd safety and pyrotechnic hazards:

Risk Factor	Risk Delta	Type
Large Attendance (Attendees > 500)	+1.5%	<i>Risk Increase</i>
Temporary Booths or Structures	+1.0%	<i>Risk Increase</i>
High-Value Equipment Assets Present	+2.0%	<i>Risk Increase</i>
Low Emergency Preparedness Level	+2.0%	<i>Risk Increase</i>
No Fire NOC Obtained (Critical Compliance Failure)	+2.5%	<i>Risk Increase</i>
CCTV Surveillance Deployed (Risk Mitigation)	-0.5%	<i>Mitigation</i>

Maximum possible Corporate Conference domain risk (all positive factors, no mitigation): +9.0%. The lower ceiling compared to the Music Concert strategy reflects the reduced crowd volatility and absence of pyrotechnic and celebrity-security risks typically associated with corporate gatherings.

4.5 Premium Calculation Formula

Once the total risk percentage (domain risk + weather risk) is established, the PolicySubscriptionService calculates the insurance premium using the following formula:

Premium = Event Budget x Policy Base Rate x (1 + Total Risk Percentage / 100)

Where Total Risk Percentage = Domain Risk % + Weather Risk %. This formula ensures that premium scales linearly with both financial exposure (event budget) and composite risk score. The Policy Base Rate represents the insurer's fundamental pricing factor for the policy domain, expressed as a decimal fraction (e.g., 0.05 for 5%).

4.6 WeatherRiskService — Live Meteorological Integration

The WeatherRiskService is invoked exclusively for outdoor events (is_outdoor = true). It integrates with the WeatherAPI.com REST API to retrieve current weather conditions for the event's location. The integration target endpoint is: GET `https://api.weatherapi.com/v1/current.json?key={API_KEY}&q={location}`

The service extracts four meteorological data points from the API response JSON and applies the following conditional risk penalties:

Weather Parameter	Threshold Condition	Risk Penalty	Rationale
Temperature (degrees C)	> 45 degrees C (Extreme Heat)	+1.0%	Risk of heat exhaustion and infrastructure stress at outdoor venues
Wind Speed (km/h)	> 20 km/h (High Wind)	+1.0%	Risk of structural damage to temporary stages and erected equipment
Humidity (%)	> 80% (High Density)	+0.5%	Contributes to equipment failure risk and attendee health concerns
Precipitation (mm)	> 5mm (Rain Alert)	+2.0%	Highest weather risk — causes slip hazards, equipment damage, and event disruption

The maximum possible weather risk addition is +4.5% (all four thresholds exceeded simultaneously). The WeatherRiskService handles API failures gracefully; if the external call fails or times out, the weather risk component defaults to 0.0% and the subscription workflow continues without interruption. The raw API data (temperature, wind, humidity, precipitation values) is persisted on the PolicySubscription entity for Underwriter review transparency.

4.7 Risk Engine Input/Processing/Output Specification

4.7.1 Input Specification

The risk engine receives an Event entity object populated from the customer's event creation form. The entity includes all risk-relevant boolean fields, the attendee count integer, the `is_outdoor` boolean, and the location string for weather API queries. The `event_type` enum value is used to select the appropriate risk strategy class at the start of computation.

4.7.2 Processing Steps

The `RiskCalculationService` first reads the event type to select the strategy. It instantiates the strategy and invokes `calculateRisk(event)`, which evaluates each defined risk factor, accumulates the domain risk percentage, and returns the result as a double. Concurrently, if `is_outdoor` is true, the `WeatherRiskService` fetches live weather data for the event location, applies the threshold evaluations, and returns the weather risk component. The two results are summed to produce the total risk percentage.

4.7.3 Output Specification

The risk engine outputs three values that are persisted on the `PolicySubscription` entity: `risk_percentage` (domain risk component), `weather_risk` (meteorological component), and `total_risk` (sum). Additionally, the calculated `premium_amount` is computed and stored. These values are presented to the Underwriter during review and to the Customer in their subscription detail view, providing full transparency into how the premium was derived.

Section 5: Data Design and Persistence Strategy

5.1 Entity Relationship Overview

The Event Rakshak data model consists of six primary entity types organized into a relational structure that faithfully represents the insurance domain. The entities are interconnected through well-defined foreign key relationships enforced at the JPA/Hibernate ORM layer and at the H2 database constraint level.

Figure 8: Event Rakshak Entity Relationship Diagram (ERD)

Full Event Rakshak Data Model — Entities: USERS, ROLES, EVENTS, RISKS, POLICIES, POLICY_SUBSCRIPTIONS, CLAIMS, NOTIFICATIONS, USER_ROLE

5.2 Entity Specifications

5.2.1 User Entity (users table)

The User entity is the foundational identity record for all participants in the Event Rakshak system. It stores authentication credentials and profile information and serves as the anchor for all role-based access control decisions.

Field Name	Data Type	Constraints	Description
user_id	BIGINT	PK, Auto-Increment, NOT NULL	Surrogate primary key — system-generated unique identifier
full_name	VARCHAR(255)	NOT NULL	The user's complete legal or display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Email address — serves as the login credential and unique user identifier
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password — never stored or transmitted in plain text
company_name	VARCHAR(255)	NULLABLE	The organization or company the user represents
phone_number	VARCHAR(20)	NULLABLE	Contact phone number for account communication
is_enabled	BOOLEAN	DEFAULT TRUE, NOT NULL	Account activation flag — disabled accounts cannot authenticate
created_at	TIMESTAMP	NOT NULL	Record creation timestamp for audit trail

The User entity participates in a Many-to-Many relationship with the Role entity through a user_roles bridge table. JPA annotation: `@ManyToMany(fetch = FetchType.EAGER)`. Eager fetch ensures roles are available immediately upon principal loading by Spring Security's CustomUserDetailsService.

5.2.2 Role Entity (roles table)

The Role entity defines the authorization levels available in the system. The four roles — ADMIN, CUSTOMER, UNDERWRITER, CLAIMS_OFFICER — correspond to the four distinct user personas. Roles are pre-seeded by the data.sql initialization script and are referenced by the JWT token generation logic to populate the granted authorities collection in the Spring Security authentication token.

5.2.3 Event Entity (events table)

The Event entity is the central business object of the Event Rakshak domain model. It captures the complete operational metadata of an insurable event and provides the inputs for the Risk Assessment Engine. The entity has a Many-to-One relationship with User (the owning Customer).

Field Name	Data Type	Constraints	Description
event_id	BIGINT	PK, Auto-Increment	Surrogate primary key
event_name	VARCHAR(255)	NOT NULL	Descriptive name of the event
event_type	VARCHAR(50)	NOT NULL	Enum: MUSIC_CONCERT or CORPORATE_CONFERENCE
event_date	DATE	NOT NULL	Scheduled date — used for weather API queries
location	VARCHAR(255)	NOT NULL	Venue location city — used for weather API geographic parameter
budget	DECIMAL(15,2)	NOT NULL	Total event budget in INR — base for premium calculation
attendees	INTEGER	NOT NULL	Expected attendee count — threshold factor for crowd risk
is_outdoor	BOOLEAN	NOT NULL	Outdoor flag — triggers WeatherRiskService if true
venue_type	VARCHAR(100)	NULLABLE	Descriptive venue type (e.g., Stadium, Convention Center)
user_id	BIGINT	FK to users.user_id, NOT NULL	Owner Customer reference
(12 risk boolean fields)	BOOLEAN x12	NULLABLE	All risk-relevant event characteristic flags per Section 3.1.3

5.2.4 Policy Entity (policies table)

The Policy entity represents an insurance product template defined by the Administrator. Policies are not directly associated with customers; they are referenced through PolicySubscription records.

Field Name	Data Type	Constraints	Description
policy_id	BIGINT	PK, Auto-Increment	Surrogate primary key
policy_name	VARCHAR(255)	NOT NULL, UNIQUE	Unique commercial name of the policy product
description	TEXT	NULLABLE	Full textual description of policy terms and coverage
domain	VARCHAR(50)	NOT NULL	Applicable event domain: MUSIC or CORPORATE
base_rate	DECIMAL(5,4)	NOT NULL	Base pricing rate as decimal fraction (e.g., 0.05 = 5%)
max_coverage	DECIMAL(15,2)	NOT NULL	Maximum claim payout limit in INR
covers_theft	BOOLEAN	NOT NULL	Whether theft incidents are covered under this policy
covers_weather	BOOLEAN	NOT NULL	Whether weather-related incidents are covered
covers_fire	BOOLEAN	NOT NULL	Whether fire-related incidents are covered
covers_cancellation	BOOLEAN	NOT NULL	Whether event cancellation is covered

5.2.5 PolicySubscription Entity (policy_subscriptions table)

The PolicySubscription entity is the operational record that binds a Customer's Event to a Policy product and tracks the complete underwriting lifecycle. It is the most attribute-rich entity in the system, holding computed risk values, financial figures, and workflow state.

Field Name	Data Type	Constraints	Description
subscription_id	BIGINT	PK, Auto-Increment	Surrogate primary key
event_id	BIGINT	FK to events.event_id, NOT NULL	Reference to the covered event
policy_id	BIGINT	FK to policies.policy_id, NOT NULL	Reference to the selected policy template
assigned_underwriter_id	BIGINT	FK to users.user_id, NULLABLE	Reference to the assigned Underwriter user
risk_percentage	DECIMAL(5,2)	NOT NULL	Domain risk score from RiskCalculationService
weather_risk	DECIMAL(5,2)	NOT NULL, DEFAULT 0	Weather risk component from WeatherRiskService

Field Name	Data Type	Constraints	Description
total_risk	DECIMAL(5,2)	NOT NULL	Combined risk: risk_percentage + weather_risk
premium_amount	DECIMAL(12,2)	NOT NULL	Calculated or underwriter- overridden premium in INR
status	VARCHAR(50)	NOT NULL	Workflow state: PENDING_REVIEW, UNDER_REVIEW, PAYMENT_PENDING, ACTIVE, REJECTED
is_paid	BOOLEAN	NOT NULL, DEFAULT FALSE	Payment completion flag — set true on successful payment
underwriter_notes	TEXT	NULLABLE	Underwriter comments, override justifications, or rejection reasons
created_at	TIMESTAMP	NOT NULL	Application submission timestamp
updated_at	TIMESTAMP	NOT NULL	Last status update timestamp — updated on every state transition

5.2.6 Claim Entity (claims table)

The Claim entity records an insurance claim filed against an active PolicySubscription. It captures the incident details, requested amount, documentary evidence path, and adjudication outcome.

Field Name	Data Type	Constraints	Description
claim_id	BIGINT	PK, Auto-Increment	Surrogate primary key
subscription_id	BIGINT	FK to policy_subscriptions, NOT NULL	Parent subscription reference
assigned_officer_id	BIGINT	FK to users.user_id, NULLABLE	Assigned Claims Officer reference
description	TEXT	NOT NULL	Detailed narrative description of the incident and claimed loss
claim_amount	DECIMAL(12,2)	NOT NULL	Amount requested by the customer for the claimed loss
incident_date	DATE	NOT NULL	Date on which the insured incident occurred
evidence_doc_path	VARCHAR(500)	NULLABLE	File system path to the uploaded evidence document

Field Name	Data Type	Constraints	Description
status	VARCHAR(50)	NOT NULL	PENDING, APPROVED, or REJECTED
approved_amount	DECIMAL(12,2)	NULLABLE	Actual payout amount approved by the Claims Officer
rejection_reason	TEXT	NULLABLE	Mandatory explanation when claim status is REJECTED
created_at	TIMESTAMP	NOT NULL	Claim submission timestamp
updated_at	TIMESTAMP	NOT NULL	Last adjudication update timestamp

5.2.7 Notification Entity (notifications table)

The Notification entity stores system-generated alerts for users, enabling an audit trail of workflow events and providing the data source for the real-time notification center in the UI.

Field Name	Data Type	Constraints	Description
id	BIGINT	PK, Auto-Increment	Surrogate primary key
user_id	BIGINT	FK to users.user_id, NOT NULL	Recipient user reference
message	TEXT	NOT NULL	Human-readable notification message
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	Read/unread flag for badge count display in frontend
timestamp	TIMESTAMP	NOT NULL	Creation timestamp for chronological ordering in UI

5.3 Database Normalization Strategy

The Event Rakshak schema adheres to Third Normal Form (3NF) normalization principles. Each table represents a single entity with no repeating groups (1NF), no partial functional dependencies on composite keys (2NF), and no transitive functional dependencies between non-key attributes (3NF). The user_roles bridge table implements the Many-to-Many User-Role relationship without introducing update anomalies. The choice to store risk factor flags as individual boolean columns on the Event entity (rather than a normalized EAV or JSON structure) was made deliberately to facilitate direct JPQL query predicates in risk strategy implementations.

5.4 Initial Seed Data Catalog

The data.sql initialization script populates the database with the following baseline records upon first deployment:

- 4 Role records: ADMIN, CUSTOMER, UNDERWRITER, CLAIMS_OFFICER
- 1 Default Admin user: admin@event rakshak.com (password: admin123 BCrypt-hashed)

- 5 Music Concert policies: Base rates ranging from 0.03 to 0.08; maximum coverage INR 5,00,000 to INR 50,00,000
- 5 Corporate Conference policies: Base rates ranging from 0.025 to 0.07; maximum coverage INR 10,00,000 to INR 1,00,00,000

Section 6: Non-Functional Requirements

6.1 Security Requirements

6.1.1 Authentication Security

All user passwords must be hashed using the BCrypt algorithm with a minimum work factor (cost parameter) of 10. Plain-text password storage or retrieval is strictly prohibited at every layer. The password field on the User entity must be excluded from all API response serialization using `@JsonIgnore` or equivalent annotations. JWT tokens must be signed using an application-specific secret key of at least 256 bits, externalized to application configuration and managed via environment variables in production. Token expiration must be enforced on every request; the backend must reject expired tokens with a 401 response even if the signature is otherwise valid.

6.1.2 Role-Based Access Control (RBAC)

The RBAC model is enforced at two distinct layers: Frontend Route Guards (Angular RoleGuard intercepts navigation and reads role from decoded JWT) and Backend Method Security (Spring Security `@PreAuthorize` annotations enforce role requirements at the API layer). The backend enforcement is the authoritative security boundary. Data isolation is also enforced: customer queries for events and subscriptions are filtered server-side by the authenticated user's `user_id`, preventing cross-customer data access regardless of URL parameter manipulation attempts.

6.1.3 Input Validation

All user-supplied data is validated at both the frontend (Angular Reactive Forms validators) and the backend (Spring's `@Valid` annotation and Bean Validation constraints on DTOs). Backend validation is authoritative. SQL injection prevention is inherently provided by JPA's parameterized query model. Raw SQL string construction must not be used in any service or repository method.

6.2 Performance Requirements

API Operation	Max Response Time	Notes
POST /api/auth/login	< 500ms	Includes BCrypt password verification overhead
POST /api/events	< 300ms	Pure database write — no external calls
POST /api/subscriptions (risk calc)	< 2,000ms	Includes optional WeatherAPI external call (~1,000ms budget)
GET /api/policies	< 200ms	Read-only catalogue — should be highly responsive
PUT /api/underwriter/** (approval)	< 400ms	Database write + notification creation
POST /api/claims	< 500ms	Includes document path persistence
GET /api/admin/metrics	< 1,000ms	Aggregate queries — may require index optimization

For WeatherAPI integration, the external HTTP call must be wrapped in a configurable timeout (recommended: 5 seconds). Failures beyond this threshold must be handled gracefully, defaulting weather risk to 0.0% and logging a warning for operational visibility.

6.3 Availability and Reliability

In the target development and demonstration environment, the system is expected to maintain availability during business hours with no formal SLA. For production deployment, the following reliability requirements apply:

- Structured logging using SLF4J with Logback to capture operational events, errors, and external API call results.
- H2 file-based persistence provides basic durability but does not support replication. Production deployments requiring higher availability should evaluate migration to PostgreSQL or MySQL.
- The WeatherAPI integration must implement circuit-breaker semantics in production to prevent cascading failures if the external service becomes unavailable.

6.4 H2 Database — Disaster Recovery Considerations

The H2 database operates in file-based persistence mode. The `insurance_db` files should be stored in an explicitly defined, version-controlled directory path for predictable recovery. Regular filesystem-level backups of the H2 database files (`insurance_db.mv.db`) must be performed. The `data.sql` script must use idempotent INSERT patterns to prevent duplicate seed data on application restart. For production schema management, adopting Flyway or Liquibase is recommended over relying solely on Hibernate `ddl-auto=update`.

6.5 Scalability and Maintainability

The layered architecture enables horizontal scaling by distributing request handling across multiple Spring Boot instances behind a load balancer, provided JWT authentication remains stateless. The Strategy pattern in the Risk Engine supports new event domains without modifying existing code (Open/Closed Principle). The RESTful API contract enables future mobile clients or third-party integrations without backend changes. Angular's component-based architecture enables independent feature module updates.

6.6 Usability Requirements

- Responsive Design: Fully functional on screen resolutions from 375px (mobile) to 1920px (desktop).
- Form Validation: All validation errors displayed inline, adjacent to the offending field, with specific and actionable messages.
- Loading States: All API calls exceeding 200ms must display a loading indicator.
- Empty States: All list views must handle empty data sets with contextual messages and call-to-action prompts.
- Accessibility: Form inputs associated with accessible labels; color-coding supplemented with text labels or icons for color-blind users.

Appendix A: Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend Framework	Angular	19+	SPA framework for all UI components and routing
Frontend Styling	CSS (Modular)	Component-scoped	Component-level style encapsulation
Frontend Security	JWT Interceptors + RoleGuard	Custom	Token injection and route authorization
Backend Framework	Spring Boot	3.4.x	RESTful API framework and dependency injection container
Backend Build	Apache Maven	3.x	Dependency management and build automation
Backend Security	Spring Security + JWT	6.x	Authentication filter chain and RBAC enforcement
ORM / Persistence	Spring Data JPA / Hibernate	6.x	Entity management, JPQL, schema management
Database	H2 Database Engine	2.x (File mode)	Persistent relational data store for development/demo
API Documentation	SpringDoc OpenAPI (Swagger)	2.x	Auto-generated interactive API documentation at /swagger-ui
External Integration	WeatherAPI.com	v1 (current.json)	Real-time meteorological data for outdoor event risk

Appendix C: Complete API Endpoint Reference

HTTP Method	Endpoint Path	Required Role	Description
POST	/api/auth/login	Public	Authenticate with email and password; returns signed JWT
POST	/api/auth/register	Public	Register new Customer account with profile data
GET	/api/events	CUSTOMER	List all events belonging to the authenticated customer

HTTP Method	Endpoint Path	Required Role	Description
POST	/api/events	CUSTOMER	Create a new event record with full metadata
GET	/api/events/{id}	CUSTOMER	Retrieve a specific event (ownership enforced server-side)
PUT	/api/events/{id}	CUSTOMER	Update event details (allowed only pre-subscription)
DELETE	/api/events/{id}	CUSTOMER	Delete an event (allowed only pre-subscription)
GET	/api/policies	CUSTOMER	Retrieve all active policy templates for the marketplace
GET	/api/policies/{id}	CUSTOMER	Retrieve a specific policy by ID with full coverage details
POST	/api/subscriptions	CUSTOMER	Apply for insurance — triggers automated risk calculation
GET	/api/subscriptions	CUSTOMER	List all subscriptions for the authenticated customer
GET	/api/subscriptions/{id}	CUSTOMER	Get detailed view including risk breakdown and premium
POST	/api/payments/{subscriptionId}	CUSTOMER	Simulate premium payment for approved subscription
POST	/api/claims	CUSTOMER	File a new insurance claim with incident data and evidence
GET	/api/claims	CUSTOMER	List all claims filed by the authenticated customer
GET	/api/underwriter/subscriptions	UNDERWRITER	Get queue of subscriptions pending review
PUT	/api/underwriter/subscriptions/{id}/approve	UNDERWRITER	Approve subscription with optional premium override

HTTP Method	Endpoint Path	Required Role	Description
PUT	/api/underwriter/subscriptions/{id}/reject	UNDERWRITER	Reject subscription with mandatory rejection reason
GET	/api/claims-officer/claims	CLAIMS_OFFICER	Get pending claims adjudication queue
PUT	/api/claims-officer/claims/{id}/approve	CLAIMS_OFFICER	Approve claim with approved payout amount
PUT	/api/claims-officer/claims/{id}/reject	CLAIMS_OFFICER	Reject claim with mandatory rejection reason
GET	/api/admin/users	ADMIN	List all registered users across all roles
PUT	/api/admin/users/{id}/status	ADMIN	Activate or deactivate a user account
POST	/api/admin/policies	ADMIN	Create new insurance policy template
PUT	/api/admin/policies/{id}	ADMIN	Update existing policy template details
DELETE	/api/admin/policies/{id}	ADMIN	Delete policy template (validated against active subscriptions)
GET	/api/admin/metrics	ADMIN	Retrieve system-wide operational metrics for dashboard
GET	/api/notifications	ALL AUTH	Retrieve notification history for the authenticated user
PUT	/api/notifications/{id}/read	ALL AUTH	Mark a specific notification as read

Section 7: Testing Specification

This section provides the complete Testing Specification for the Event Rakshak platform. It defines the unit testing philosophy, tooling, coverage expectations, a sample test registry, and annotated production-grade code specimens for both the Java backend and the Angular 21 frontend. The specification is structured to serve as a living reference for developers, QA engineers, and technical reviewers throughout the project lifecycle.

7.4 Unit Testing Specification

Unit testing forms the innermost ring of the Event Rakshak test pyramid. The goal of unit testing is to validate individual units of source code in complete isolation from their collaborators — repositories, external APIs, HTTP clients, and other services. A failing unit test must always point to a single, clearly identifiable cause within the unit under test. The project enforces a minimum branch coverage threshold of 80% for the service layer and 70% for Angular 21 components, validated automatically in the CI/CD pipeline.

7.4.1 Backend Unit Testing (Java)

The Event Rakshak backend is written in Java using the Spring framework. All unit tests use JUnit 5 (Jupiter) for test lifecycle management, Mockito for dependency isolation and behaviour verification, and AssertJ for fluent, human-readable assertion chaining. Tests are executed via the Maven Surefire plugin during every build. No running application context is loaded during unit tests — all Spring container features are deliberately excluded to keep tests fast, deterministic, and isolated. The `@SpringBootTest` annotation must never appear in a unit test class.

The testing stack: JUnit 5 provides `@Test`, `@BeforeEach`, `@ParameterizedTest`, `@MethodSource`, and `@DisplayName`. Mockito provides `@Mock` (mock collaborators), `@InjectMocks` (class under test with mocks injected), `@Captor` (argument capture), and stubbing via `when().thenReturn()` / `doThrow()` / `thenAnswer()`. AssertJ's `assertThat()` API produces clear, chainable assertions with informative failure messages.

A. Testing Philosophy — Mocking Strategy

The service layer contains all business logic and is therefore the primary target of unit testing. Services depend on multiple collaborators: JPA repositories (H2 database calls), `WeatherRiskService` (external `WeatherAPI.com` HTTP call), `PasswordEncoder`, and `JwtTokenProvider`. During unit testing, none of these collaborators should be real — doing so introduces network latency, database state management, and environmental dependencies that make tests slow and non-repeatable.

Mockito creates lightweight mock objects substituting real collaborators. The `@Mock` annotation declares a mock field. `MockitoAnnotations.openMocks(this)` in `@BeforeEach` initialises all mocks. `@InjectMocks` creates a real instance of the class under test and injects all declared mocks via constructor or field injection — the class runs its actual production code while every collaborator it calls is controllable.

- All JPA repositories (`EventRepository`, `PolicyRepository`, `ClaimRepository`, etc.) are mocked — no embedded database in unit tests.

- WeatherRiskService is always mocked in RiskCalculationService tests, returning deterministic weather data.
- PasswordEncoder (BCryptPasswordEncoder) is mocked in AuthService tests to return predictable hash values.
- JwtTokenProvider is mocked in authentication tests to return a pre-defined token string.
- The test class must NOT be annotated with @SpringBootTest or any Spring context loader.

B. Core Service Layer Testing

1. RiskCalculationService

RiskCalculationService is the most algorithmically complex service in the Event Rakshak backend. It implements the Strategy design pattern, selecting the algorithm based on the event domain (OUTDOOR_MUSIC_CONCERT or CORPORATE_Tech_CONFERENCE). Its unit test suite must validate: strategy selection correctness, individual risk factor application (penalties and mitigations), weather risk integration, and the final premium calculation formula.

Strategy Pattern Validation: Tests must verify that an Event with eventType = OUTDOOR_MUSIC_CONCERT invokes MusicConcertRiskStrategy, and eventType = CORPORATE_Tech_CONFERENCE invokes ConferenceRiskStrategy. This is verified by asserting that the returned risk percentage falls within the expected range for each domain and by using Mockito's verify() to confirm the correct strategy method was called once.

Combined Risk Score: A @ParameterizedTest with @MethodSource covers at least eight distinct event configurations varying attendee count, outdoor flag, has_fire_noc, has_metal_detectors, and has_pyrotechnics. Each test asserts $\text{total_risk} = \text{domain_risk} + \text{weather_risk}$ and $\text{premium} = \text{max_coverage} * (\text{total_risk} / 100)$ with tolerance 0.001 via isCloseTo().

- Test: Music concert, 1500 attendees, no Fire NOC, pyrotechnics → domain_risk = 8.5%.
- Test: Music concert, 800 attendees, all mitigations → domain_risk = 0.5%.
- Test: Corporate, high-value equipment, no Fire NOC → domain_risk = 6.5%.
- Test: Outdoor event, humidity > 80% + precipitation > 5mm → weather_risk = 2.5%.
- Test: Indoor event → verify(weatherRiskService, never()).getWeatherRisk(any()).

2. ClaimService

ClaimService manages the complete claim lifecycle. Unit tests must validate all permitted and prohibited state transitions, the approved-amount business rule, and notification generation. All repository calls are mocked to return pre-constructed entities.

State Transition Testing: approveClaim() on a PENDING claim must set status = APPROVED, approved_amount, resolved_at, and assigned_officer_id. approveClaim() on an already-APPROVED or REJECTED claim must throw InvalidClaimStateException. rejectClaim() on a PENDING claim must set status = REJECTED and capture rejection_reason.

Approved Amount Validation: approved_amount = 0 must throw ClaimValidationException. approved_amount > claim_amount must throw ClaimValidationException with message 'Approved amount cannot exceed claimed amount'. A valid amount must persist the entity via claimRepository.save() and save a Notification via notificationRepository.save().

3. AuthService

AuthService handles registration, credential validation, and JWT issuance. Tests must validate password encoding, JWT payload correctness, duplicate email detection, and role assignment. PasswordEncoder and JwtTokenProvider are both mocked.

Password Encoding: The mock PasswordEncoder returns '\$2a\$10\$mockHash'. The test asserts the User entity passed to userRepository.save() has password = '\$2a\$10\$mockHash' and never the raw password. Duplicate Email: registering with an existing email must throw UserAlreadyExistsException before any call to passwordEncoder.encode() or save(). JWT: The returned AuthResponse must contain the mocked token, correct email, and roles.

C. Entity and DTO Validation Testing

Jakarta Bean Validation annotations (@NotNull, @NotBlank, @Email, @Size, @Min, @Max) on DTO classes are verified using the Jakarta Validator API directly — no Spring context needed. A ValidatorFactory is obtained via Validation.buildDefaultValidatorFactory() in @BeforeAll. Tests construct a DTO with invalid fields and assert that validator.validate(dto) returns violations with the expected property paths and messages.

DTO Security: Tests must verify that UserDTO has no getPassword() method and that Jackson serialization of UserDTO produces JSON without a 'password' key. This guards against accidental password leakage in API responses.

7.4.2 Frontend Unit Testing (Angular 21)

The Event Rakshak frontend is built with Angular 21. Frontend unit tests are written in Jasmine, a behaviour-driven framework with describe/it/expect API, and executed by the Karma test runner configured with a headless Chromium instance (karma-chrome-launcher with --headless --no-sandbox flags) for CI/CD compatibility. Angular's TestBed utility provides an isolated module for each test class, enabling precise dependency injection control.

Tests are co-located with source files (component.spec.ts alongside component.ts). The angular.json test configuration specifies karma.conf.js, Istanbul coverage collection, and minimum thresholds enforced at CI time: statements 75%, branches 70%, functions 75%, lines 75%.

A. Component Testing

Angular 21 component tests use TestBed.configureTestingModule() to declare the component and provide spy objects via jasmine.createSpyObj(). After compileComponents() and fixture.detectChanges(), the test inspects the DOM via fixture.debugElement and simulates events via nativeElement.click() or triggerEventHandler().

UI State Testing: Components showing loading spinners are tested in two states: (1) isLoading = true — fixture.debugElement.query(By.css('.loading-spinner')) must be non-null; (2) after the observable emits — spinner absent, data table present. DOM Validation: Critical fields are asserted via .nativeElement.textContent. Tables: fixture.debugElement.queryAll(By.css('tbody tr')).length must equal mock data length.

B. Reactive Form Testing

Angular 21 Reactive Forms are tested by setting control values directly and asserting valid/invalid state — no DOM interaction required. Cross-field validators are tested at the form level.

- Email: 'invalid-email' → status INVALID, error 'email'. 'user@example.com' → VALID.
- Password < 8 chars → error 'minlength'. 'Password@1' → VALID.
- Password mismatch cross-field: form-level error 'passwordMismatch' must be truthy.
- Budget = -1 → error 'min'. Budget = 100000 → VALID.
- Past event date → custom error 'pastDate'. Future date → VALID.
- onSubmit() with invalid form must NOT call the service method.
- onSubmit() with valid form must call service with exact form value.

C. Service and Guard Testing

Angular 21 services making HTTP calls are tested using HttpClientTestingModule and HttpTestingController, which intercepts outbound requests, allowing assertion of URL, method, and headers before flushing a mock response.

AuthService: login() must POST to '/api/auth/login' with correct body and store the JWT in localStorage. logout() must remove the JWT from localStorage. isAuthenticated() must return false with no token and true with a valid non-expired token (tested using jasmine.clock() to simulate expiry).

RoleGuard canActivate(): (1) User with required role → returns true. (2) User without required role → returns false, router.navigate(['/unauthorized']). (3) Unauthenticated user → returns

false,router.navigate(['/login']). AuthInterceptor: outbound request must have 'Authorization: Bearer <token>' header when token is stored, and must omit the header when no token exists.

7.4.3 Sample Unit Test Registry

The following table enumerates the minimum required unit test scenarios for Event Rakshak. It serves as the QA sign-off acceptance baseline.

Layer	Component / Service	Test Scenario	Expected Result
Backend	RiskCalculationService	Music concert 1500 attendees, no Fire NOC, pyro	domain_risk = 8.5%
Backend	RiskCalculationService	Corporate conference, high-value equip, no Fire NOC	domain_risk = 6.5%
Backend	RiskCalculationService	Indoor event — WeatherRiskService not invoked	verify(weatherSvc, never()).getWeatherRisk()
Backend	RiskCalculationService	Outdoor: humidity>80% + rain>5mm	weather_risk = 2.5%
Backend	RiskCalculationService	Premium: max_coverage=500000, total_risk=8.0%	premium = 40000.00 ± 0.001
Backend	ClaimService	Approve PENDING claim, valid amount	status=APPROVED, amount saved
Backend	ClaimService	Reject PENDING claim with reason	status=REJECTED, reason saved
Backend	ClaimService	Approve already-APPROVED claim	InvalidClaimStateException thrown
Backend	ClaimService	Approve: amount > claim_amount	ClaimValidationException thrown
Backend	ClaimService	Approve PENDING → notification generated	notificationRepository.save() called once
Backend	AuthService	Register: password stored as BCrypt hash	user.password != rawPassword
Backend	AuthService	Register duplicate email	UserAlreadyExistsException thrown
Backend	AuthService	Login correct credentials → JWT returned	response.token = 'mock.jwt.token'
Backend	AuthService	JWT payload has correct email + roles	claims match input
Backend	RegistrationRequest DTO	Empty email field	ConstraintViolation on 'email'
Backend	UserDTO	Serialised JSON contains no 'password' key	JSON.has('password') = false
Frontend	LoginComponent	Submit empty form → service not called	loginService.login() NOT called
Frontend	LoginComponent	Submit valid form → service called	login() called with form value
Frontend	LoginComponent	401 response → error message shown	error div textContent = 'Invalid credentials'
Frontend	CreateEventComponent	Past event date	form error 'pastDate' truthy

Layer	Component / Service	Test Scenario	Expected Result
Frontend	SubscriptionListComponent	Loading state	spinner present before data emits
Frontend	SubscriptionListComponent	Data rendered	tbody tr count = mock array length
Frontend	AuthService (Angular 21)	login() POST endpoint	req.url contains '/auth/login'
Frontend	AuthService (Angular 21)	logout() clears token	localStorage.getItem('token') = null
Frontend	RoleGuard	CUSTOMER accessing CUSTOMER route	canActivate() = true
Frontend	RoleGuard	CUSTOMER accessing ADMIN route	false + navigate('/unauthorized')
Frontend	AuthInterceptor	Token stored → header present	Authorization = 'Bearer <token>'
Frontend	AuthInterceptor	No token → header absent	has('Authorization') = false

7.4.4 Test Code Snippets

The following production-grade code specimens are provided as implementation templates and QA acceptance criteria examples. All tests follow the AAA pattern (Arrange / Act / Assert) and use the 'should_Expected_When_State' naming convention.

Snippet 1 — JUnit 5: RiskCalculationServiceTest

Validates the Music Concert risk strategy, indoor guard, and premium formula.

```
// =====  
// RiskCalculationServiceTest.java - Event Rakshak  
// =====  
package org.eventrakshak.service;  
  
import org.eventrakshak.entity.Event;  
import org.eventrakshak.entity.Policy;  
import org.eventrakshak.enums.EventType;  
import org.eventrakshak.service.risk.WeatherRiskService;  
import org.junit.jupiter.api.*;  
import org.mockito.*;  
import static org.assertj.core.api.Assertions.*;  
import static org.mockito.ArgumentMatchers.any;  
import static org.mockito.Mockito.*;  
  
class RiskCalculationServiceTest {  
  
    @Mock  
    private WeatherRiskService weatherRiskService;  
  
    @InjectMocks  
    private RiskCalculationService riskCalculationService;  
  
    @BeforeEach  
    void setUp() { MockitoAnnotations.openMocks(this); }  
  
    private Event buildMusicEvent(int attendees, boolean fireNoc,  
                                  boolean pyro, boolean outdoor) {  
        Event e = new Event();  
        e.setEventType(EventType.OUTDOOR_MUSIC_CONCERT);  
        e.setNumberOfAttendees(attendees);  
        e.setHasFireNoc(fireNoc);  
        e.setFireworksUsed(pyro);  
        e.setIsOutdoor(outdoor);  
        e.setHasMetalDetectors(false);  
        return e;  
    }  
  
    @Test  
    @DisplayName("Music: 1500 attendees + no FireNOC + pyro = 8.5%")  
    void should_Return8Point5_When_HighRiskMusicConcert() {  
        Event event = buildMusicEvent(1500, false, true, true);  
        when(weatherRiskService.getWeatherRisk(any())).thenReturn(0.0);  
  
        double risk = riskCalculationService.calculateDomainRisk(event);  
  
        assertThat(risk).isCloseTo(8.5, within(0.001));  
    }  
  
    @Test
```